ДЗ4

Технический дизайн микросервисной архитектуры

Сервис аренды недвижимости RentEase

Выполнил: Фёдоров Иван
Группа: БР1.1
2026

ЗАДАЧА

Разделить монолитное приложение RentEase из ЛР1 на отдельные микросервисы по принципу database-per-service, не меняя пользовательский сценарий из Д33. Для каждого сервиса определить границу ответственности и владельца данных, описать взаимодействие сервисов, разделение базы данных, внутренние endpoint'ы, варианты ответов и ошибки, а также план перехода от монолита.

Исходная реализация использует Spring Boot, Spring Data JPA, JWT и Flyway. В монолите все доменные таблицы находятся в одной схеме, поэтому задача состоит в устранении общих FK и прямого доступа сервисов к чужим данным.

ХОД РАБОТЫ

1. Анализ исходного монолита

Контроллеры AuthController, PropertyController, BookingController, ContractController, MessageController, FavoriteController и ReviewController используют общую модель Domain и репозитории Spring Data JPA.

Группа	Таблицы	Функции
Пользователи	user_roles, users	регистрация, вход, роли tenant, landlord, admin
Недвижимость	properties, rental_conditions, property_photos	объявления, условия и фотографии
Бронирования	bookings	заявки, даты и статусы
Сделки	rental_contracts, payments	договоры и платежи
Коммуникация	messages	сообщения и диалоги
Отзывы	reviews	оценки участников сделки
Избранное	favorites	сохраненные объекты

2. Целевая архитектура

Клиент обращается к API Gateway. Gateway выполняет маршрутизацию, проверяет JWT, добавляет X-Request-Id и скрывает внутренние адреса. Каждый доменный сервис имеет собственную БД и отдельный набор Flyway-миграций. Между БД нет внешних ключей: сервисы передают UUID через API.

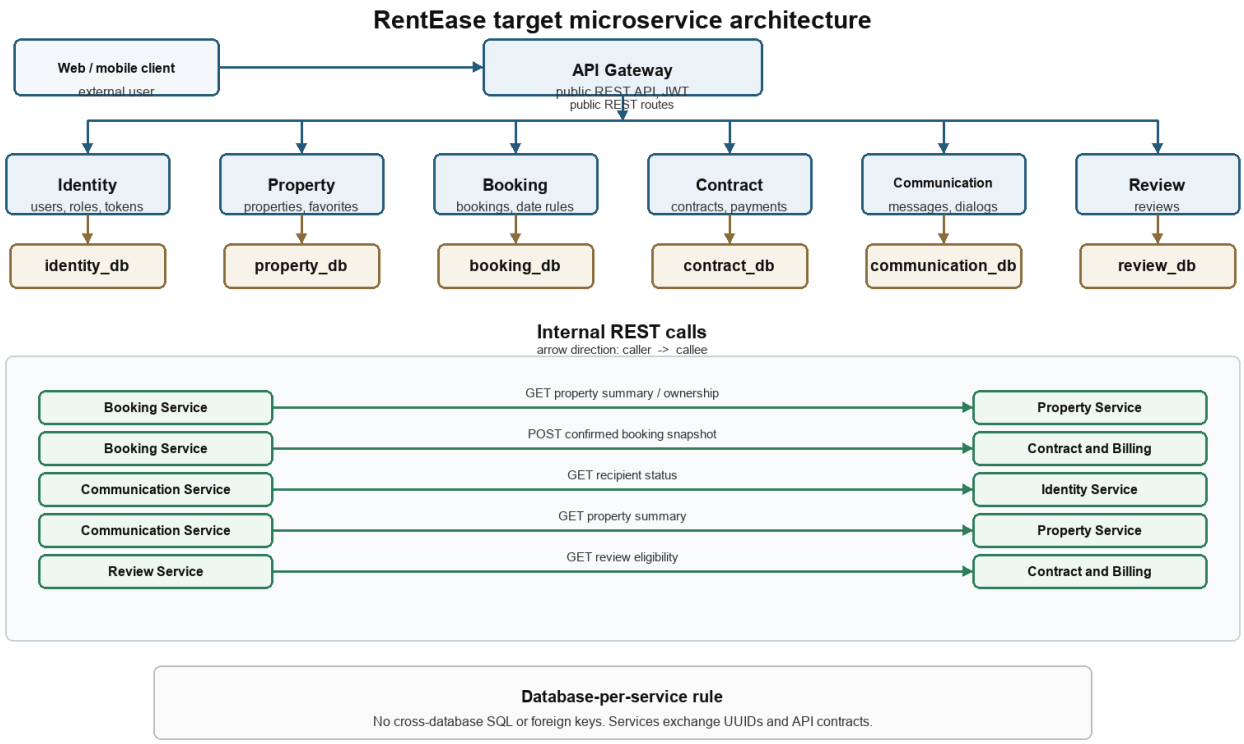


Рисунок 1 - Целевая микросервисная архитектура RentEase

3. Границы сервисов и владение данными

Сервис	База	Данные и ответственность
API Gateway	нет	единая точка входа, маршрутизация, CORS, rate limit
Identity Service	identity_db	users, user_roles, refresh_tokens; регистрация, вход, роли
Property Service	property_db	properties, conditions, photos, favorites; поиск и владелец
Booking Service	booking_db	bookings, status history; даты и жизненный цикл заявки
Contract and Billing	contract_db	rental_contracts, payments; договор и платежи
Communication Service	communication_db	messages; переписка по объекту
Review Service	review_db	reviews; отзывы участников завершенной сделки

Favorites оставлены в Property Service, поскольку это действие над объявлением и оно не требует отдельной распределенной транзакции. Внешние user_id и landlord_id хранятся как UUID без FK и проверяются через Identity Service при необходимости.

4. Взаимодействие сервисов

Для операций, где результат необходим в рамках текущего HTTP-запроса, используется внутренний REST поверх HTTP/JSON. Внутренние endpoint'ы доступны только в приватной сети и требуют сервисный bearer JWT.

Инициатор	Получатель	Вызов
Booking Service	Property Service	активность, цена и владелец
Booking Service	Contract and Billing	создание договора
Communication Service	Identity Service	проверка получателя
Communication Service	Property Service	проверка объекта
Review Service	Contract and Billing	проверка права на отзыв

4.1. Создание и подтверждение бронирования

- Tenant отправляет POST /api/v1/bookings в Gateway.

- Gateway проверяет JWT и направляет запрос в Booking Service.

- Booking Service вызывает GET /internal/properties/{property_id}/summary.

- Booking Service проверяет пересечение дат в собственной booking_db и сохраняет заявку со статусом pending.

- Landlord меняет статус; Booking Service проверяет владельца через Property Service.

- При подтверждении Booking Service отправляет POST /internal/contracts со снимком сделки.

- Contract and Billing Service идемпотентно создает договор в contract_db.

```
GET /internal/properties/{property_id}/summary
Authorization: Bearer <service-jwt>

200 OK
{
  "property_id": "d4162366-8ce4-40f0-84ad-5fc289c1ee36",
  "landlord_id": "f3f1d7d2-0a1c-4c2b-9a4e-111111111111",
  "active": true,
  "monthly_rent": 65000
}
```

4.2. Создание отзыва

Review Service вызывает GET /internal/contracts/{contract_id}/review-eligibility?author_id=... и сохраняет отзыв только при eligible=true. Повторная отправка приводит к 409 REVIEW_ALREADY_EXISTS.

5. Разделение базы данных

База	Таблицы	Внешние UUID-ссылки без FK
identity_db	user_roles, users, refresh_tokens	нет
property_db	properties, rental_conditions, property_photos, favorites	landlord_id, favorites.user_id
booking_db	bookings, booking_status_history	tenant_id, property_id
contract_db	rental_contracts, payments	booking_id, property_id, tenant_id, landlord_id
communication_db	messages	sender_id, receiver_id, property_id
review_db	reviews	contract_id, author_id, property_id

Внутри одной базы локальные связи сохраняются обычными внешними ключами. Между базами FK не создаются, чтобы сервис мог мигрировать и развиваться независимо. Каждый сервис получает собственный каталог db/migration и отдельную историю Flyway.

6. Внутренняя OpenAPI-спецификация

Полный контракт межсервисных endpoint'ов вынесен в internal-openapi.yaml. В него включены проверки пользователя, снимок объекта, проверка владельца, создание договора и проверка возможности оставить отзыв. Проверка пересечения дат остается локальной операцией Booking Service.

Endpoint	Назначение	Ответы
GET /internal/users/{user_id}/status	проверить пользователя	200, 401, 403, 404
GET /internal/properties/{property_id}/summary	снимок объекта	200, 401, 403, 404
GET /internal/properties/{property_id}/ownership	проверить владельца	200, 401, 403, 404
POST /internal/contracts	создать договор идемпотентно	201, 200, 400, 401, 403, 409, 503
GET /internal/contracts/{contract_id}/review-eligibility	проверить право на отзыв	200, 401, 403, 404

```
POST /internal/contracts
Authorization: Bearer <service-jwt>
Idempotency-Key: booking-d4162366-confirmed

{
  "booking_id": "11111111-1111-1111-1111-111111111111",
  "property_id": "d4162366-8ce4-40f0-84ad-5fc289c1ee36",
  "monthly_rent": 65000
}
```

7. Ошибки и надежность

HTTP	Код	Причина
400	INVALID_REQUEST	нет поля, неверный UUID или даты
401	SERVICE_UNAUTHORIZED	токен отсутствует или недействителен
403	SERVICE_FORBIDDEN	нет разрешения
404	*_NOT_FOUND	сущность не найдена
409	DATES_UNAVAILABLE / *_ALREADY_EXISTS	конфликт состояния или повтор
422	BUSINESS_RULE_VIOLATION	нарушено бизнес-правило
502/503	DEPENDENCY_UNAVAILABLE	зависимость недоступна

Единый ответ ошибки содержит code, message, trace_id и необязательный details. SQL, stack trace, пароли и токены клиенту не возвращаются.

8. План перехода от монолита

- Зафиксировать публичные контракты Д32 и сценарий Д33, добавить X-Request-Id.

- Запретить новые прямые обращения модулей к таблицам чужих будущих сервисов.

- Поднять отдельные базы и выполнить независимые Flyway-миграции Identity и Property Service.

- Скопировать users и роли в identity_db, затем properties, условия, фотографии и избранное в property_db; сверить записи и контрольные суммы.

- Вынести Gateway и Identity Service, сохранив публичные URL /api/v1/auth/*.

- Вынести Property, Booking, Contract, Communication и Review Service с feature flags и проверкой сценария Д33.

- Переключить трафик на Gateway, наблюдать метрики, затем удалить доступ монолита к вынесенным таблицам.

9. Нефункциональные требования

- короткоживущий access JWT и ротация refresh-токенов;

- TLS на внешнем контуре и mTLS либо service JWT внутри;

- таймаут внутренних вызовов 1-2 секунды и повторы только для безопасных GET;

- идемпотентность создания договора;

- логи с trace_id, health/readiness и метрики;

- контрактные тесты OpenAPI и независимое резервное копирование БД.

10. Критерии приемки

- единственный владелец каждой таблицы;

- нет межбазовых FK и прямого SQL;

- описаны потоки бронирования и отзыва;

- internal-openapi.yaml содержит endpoint'ы, схемы и ошибки;

- план миграции имеет контрольные точки и откат;

- сценарий Д33 остается совместимым.

ВЫВОД

Для RentEase предложено разделение монолита на API Gateway и шесть доменных сервисов. Каждый сервис имеет собственную базу и Flyway-миграции, а связи между доменами представлены UUID и внутренними REST-контрактами. Проверки активности объекта, владельца и права на отзыв выполняются через явные endpoint'ы, пересечение дат проверяется владельцем booking_db, а создание договора защищено идемпотентностью.

Такое разделение сохраняет сценарий Д33 и дает понятный путь к реализации ЛР2: сначала вынести Identity и Property, затем Booking, Contract, Communication и Review, проверяя контрактные тесты и отсутствие доступа к чужим БД после каждого шага.

Приложение. Файлы решения

Файл	Содержимое
README.md	архитектура, БД, потоки, ошибки и миграция
internal-openapi.yaml	OpenAPI 3.0.3 спецификация внутренних endpoint'ов
architecture.mmd	исходник диаграммы Mermaid
architecture.svg	векторная схема архитектуры